Sapienza Università di Roma
MSc in Engineering in Computer Science
and Artificial Intelligence
Laboratory of Advanced Programming 2025/2026
Hackathon exam - March 28, 2026

1 Mission briefing: "A Fragile Balance of Power."

The year is **2038**. The global geopolitical landscape is once again marked by tension, secrecy, and mutual distrust. Even allied nations are reluctant to share sensitive information, especially when it concerns strategic or military activity.

In response, covert operations have deployed **seismic surveillance devices** across strategic locations worldwide. These devices (hereafter referred to as *sensors*) operate in remote and sensitive areas, continuously monitoring ground vibrations.

The intelligence derived from these **signals** is operationally critical. Early detection of threats enables military command to react in time, placing air defense systems on standby, initiating defensive protocols, or escalating readiness levels.

A command center has been established in a **neutral region**. Due to its neutral status, it is strictly prohibited to host cyber services that directly process intelligence data. Only lightweight services that exclusively forward or route data/requests are permitted.

Fortunately, the nation has access to multiple geographically distributed data centers, but they may be abruptly destroyed due to conflict, sabotage, or environmental conditions. Your task is to design and implement a system that can operate reliably under these constraints, providing **continuous intelligence** to the command center even in the presence of **partial system failure**.

The fate of the nation lies in your hands.

2 General information

- **Duration:** 4 days
- **Team size:** 2 to 5 students
- **Submission deadline:** March 31, 2026, 23:59
- **Final discussion:** In-person presentation + demo (~20 minutes total per group). All members must attend.

Suggested internal milestones:

- Day 1: short pitch + user stories draft
- Day 2: architecture defined and event schema defined
- Day 3: full end-to-end system running via docker compose
- Day 4: documentation and slides finalized

3 What you are provided with

You will receive a **Docker container** simulating a set of seismic sensors. You **must not modify** the provided container.

The simulator exposes a real-time stream of seismic measurements coming from geographically distributed sensors.

3.1 How to run the simulator

To start the simulator, first load the image from the provided tar archive and then run the container by publishing port 8080:

```
docker load -i seismic-signal-simulator-oci.tar
docker run --rm -p 8080:8080 seismic-signal-simulator:multiarch_v1
```

All endpoints are served from the base URL `http://localhost:8080`. The OpenAPI documentation is available at `http://localhost:8080/docs` and `http://localhost:8080/openapi.json`. You can also verify the simulator and list available devices using:

```
curl -s http://localhost:8080/health
```

Important: in order to support all architectures, instead of providing an image per architecture, we are providing a multiarch containerd image. Check `https://docs.docker.com/desktop/features/containerd` to learn how to use containerd images on your local machine with Docker Desktop.

3.2 Seismic signal stream

Devices are discoverable via the devices endpoint:

```
curl -s http://localhost:8080/api/devices/ | jq
```

Sensor measurements are exposed **only via WebSocket** (SSE support for sensor streams is not part of the current contract). Subscribe using:

```
WS /api/device/{sensor_id}/ws
```

Each stream message is a time-domain measurement with UTC timestamp and value in `mm/s`. The default sampling frequency is **20 Hz**, configurable through `SAMPLING_RATE_HZ`.

Additional info can be found in `API_CONTRACT.md`.

3.3 Replica control stream (failure simulation)

For the purpose of this exam, **only** the processing service replicas are subject to failure. **All other components of the system can be assumed to be reliable**.

Each processing replica must listen to a control stream exposed by the simulator container. This control stream is delivered via Server-Sent Events (SSE) and may emit shutdown commands at random times to simulate node failures.

Control stream endpoint:

```
GET /api/control
```

SSE event types exposed by the simulator are `control-open`, `heartbeat`, and `command`. When a shutdown is triggered, exactly one connected listener receives the shutdown command.

When a replica receives the command:

```
{"command":"SHUTDOWN"}
```

it must terminate itself (forced shutdown).

Additional info can be found in `API_CONTRACT.md`.

3.4 Development tools

To aid the development, the container has some configurable parameters and some API calls to allow manual triggering of events. All related info can be found in `DOCKER_CONTRACT.md`, `API_CONTRACT.md`, and `http://localhost:8080/docs`.

4 What you must deliver

You must design and implement a **distributed, fault-tolerant seismic analysis platform** that:

1. Ingests seismic data from the provided simulator.
2. Redistributes each measurement across replicated processing services.
3. Performs frequency-domain analysis and event classification.
4. Persists detected events with duplicate-safe/idempotent behavior.
5. Provides a real-time dashboard for monitoring and exploration.

4.1 Custom broker service

You must implement a custom broker (fan-out component) that:

- Capture incoming measurements.
- Redistribute incoming measurements to multiple (some or all, according to your design) processing replicas.

The broker is responsible only for data distribution and must not perform any form of data processing.

For this exam, students can assume that this broker is hosted in the neutral region and does not require replication.

The baseline expectation is that all replicas receive the same stream of events (broadcast model). Partitioning of sensors across replicas is allowed as an optional design choice, provided that the system remains fault tolerant.

4.2 Processing service (replicated)

The processing service must run with multiple replicas.

Each replica must:

1. Maintain an in-memory sliding window of recent samples for each sensor.
2. Apply a Discrete Fourier Transform (DFT) or equivalent FFT-based method.
3. Extract dominant frequency components.
4. Classify events based on frequency analysis.

Use the following frequency bands for event classification (based on the dominant frequency component):

- **Earthquake:** $0.5 \leq f < 3.0$ Hz
- **Conventional explosion:** $3.0 \leq f < 8.0$ Hz
- **Nuclear-like event:** $f \geq 8.0$ Hz

Disclaimer: these frequency references are fictional and are provided only for the purpose of this exam. They do not represent validated real-world seismology thresholds.

4.3 Persistence

Detected events must be stored in a shared persistence layer.

Persistence can be implemented using any relational database (e.g., Postgres) or noSQL database (e.g., MongoDB). No embedded database (e.g., SQLite).

Since replicas may process equivalent inputs, the system must prevent duplicate event persistence.

5 Functional requirements

5.1 Gateway and fault tolerance

The system must expose a single entry point between dashboard and backend services. Students are free to choose the most appropriate implementation strategy. The chosen entry-point component should:

1. Route requests to available processing replicas.
2. Use health checks to detect unavailable replicas.
3. Exclude failed replicas automatically.

Failure of some replicas must **not** interrupt overall system operation.

5.2 Frontend dashboard

The frontend must provide a real-time monitoring dashboard.

- Real-time visualization of detected events.
- Historical event inspection.
- Optional filtering (e.g., by sensor, type, or location).

Real-time updates must be delivered via WebSocket, SSE or via REST polling.

6 Architectural constraints

Mandatory

- Use a distributed architecture (avoid a single monolithic service).
- Replicate processing services.
- Implement duplicate-safe persistence behavior.
- Provide architecture diagrams

Allowed in this exam scope: Tight coupling between services is acceptable when clearly justified in the project documentation.

7 Technical requirements

7.1 Tech stack

- Backend: any language/framework.
- Messaging: external messaging systems (e.g., Kafka, RabbitMQ) are not required.
- Database: any relational or noSQL. No embedded or in-memory databases.

- Frontend: any framework/library.

7.2 Containerization

You must provide:

- A Dockerfile per service
- A single `docker-compose.yml` that starts all services and simulator container.

The system must start with `docker compose up`. No manual setup after startup.

7.3 Baseline

All groups are expected to meet the following baseline requirements:

1. Distributed architecture
2. Fault tolerance
3. Replicated processing services (only component subject to failure)
4. Sliding-window processing + DFT/FFT analysis
5. Event classification based on frequency ranges
6. Duplicate-safe persistent event storage
7. Real-time dashboard
8. Docker reproducibility

7.4 Tasks by group size

Groups of 5 are expected to deliver the full system and implement about 30 user stories.
Groups of 4 are expected to deliver the full system and implement about 25 user stories.
Groups of 3 are expected to deliver the full system and implement about 20 user stories.
Groups of 2 are expected to deliver the full system and implement about 15 user stories.

Note: user stories must be documented in `input.md`.

8 Deliverables

Submit a public GitHub repository containing:

- Full source code
- Dockerfiles, `docker-compose.yml`
- `input.md`
- Student_doc.md
- Architecture diagrams

- User stories + LoFi mockups
- Presentation slides

The repository must have the following structure:

```
.
├── input.md
├── Student_doc.md
├── source/
└── booklets/
```

All the textual documents must follow the Markdown syntax. Please refer to `https://www.markdownguide.org/basic-syntax/` if you are not entirely familiar with it.

Following is the list of textual files to be submitted:

- `input.md` describes the system overview, the user stories, the standard event schema and the rule model. A (partial) example is accessible at `https://drive.google.com/file/d/1klvQ8r8WKfW86HS8HQ801QZh-yXZ05sK/view?usp=sharing`.
- `Student_doc.md`, which contains the specifics of the deployed system. Structure of the file: `https://drive.google.com/file/d/1stCQoen6ojT3hBexAkyp0Ja8H6XzOuFn/view?usp=sharing`. An example is accessible at `https://drive.google.com/file/d/15lmOqwYTG4qORk3bZgwMlRBWLaA5Gn-I/view?usp=sharing`.

The `booklets` folder contains whatever document you consider appropriate (slides, images, etc.).

The `source` folder contains the system developed in whatever technologies/framework you may want. In this folder all the source code, configuration files, and any other file you may need (please remember we have adopted an IaC approach - Infrastructure as Code) AND the Dockerfiles, docker-compose files, etc. which will allow the instructors to re-build/re-deploy your system on whatever platform (either on-premise or on cloud).

Each group needs to create a repo named `<MATRICOLA>_<PROJECT>` where `<MATRICOLA>` is the INFOSTUD Student ID of the group leader and `<PROJECT>` is the acronym of your project/group.

In order to avoid misunderstandings, this is repeated. Before the discussion, you need to share with the instructor all the documents and materials of your project, in particular the requirements (as user stories) of your entire system. User stories can be documented by using a spreadsheet and collecting all of them in a booklet. For each user story, it should be provided also a LoFi mockup, to be prepared with whatever tool (e.g., Balsamiq, Figma, etc.), and a textual description highlighting specific non functional requirements (if any).

The system is developed in whatever technologies/framework you may want. The release should be done by providing the link of a GitHub repo with all the source code, configuration files, any other file you may need (please remember we have adopted a IaC approach - Infrastructure as Code) AND the Dockerfiles, docker-compose files, etc. which will allow the instructors to re-build/re-deploy your system on whatever platform (either on-premise or on cloud).